

RELATÓRIO DO PROJETO – OBS ACT COMPILADOR

O que foi implementado

O projeto consistiu no desenvolvimento de um analisador sintático e compilador/transpilador para a linguagem ObsAct, uma linguagem voltada para automação e controle de dispositivos.

O sistema recebe como entrada um programa escrito na linguagem ObsAct e gera como saída um código equivalente em Python, utilizando uma arquitetura baseada em Árvore Sintática Abstrata (AST).

Requisitos obrigatórios implementados:

- Declaração de dispositivos e associação com variáveis internas.
- Inicialização automática de sensores com valor zero quando não atribuídos.
- Suporte a comandos de atribuição (set).
- Suporte a ações básicas:
 - ligar dispositivo
 - desligar dispositivo
 - verificar estado do dispositivo
- Implementação de estruturas condicionais:
 - se ... entao
 - se ... entao ... senao
- Suporte a expressões lógicas com operadores:
 - >, <, >=, <=, ==, !=
 - operador lógico &&
- Implementação de envio de alertas para dispositivos.
- Suporte a envio de broadcast, permitindo enviar alertas para múltiplos dispositivos simultaneamente.
- Geração de código final em Python com estrutura equivalente ao programa original.

Funcionalidades adicionais implementadas:

- **Uso de Árvore Sintática Abstrata (AST) para representar a estrutura do programa.**
- **Separação em módulos:**
 - **Parser (Bison)**
 - **Lexer (Flex)**
 - **AST (estrutura de dados)**
 - **Code Generator (geração de Python)**
- **Geração de código Python com indentação correta baseada na estrutura da AST.**
- **Suporte a blocos de comandos dentro de estruturas condicionais.**
- **Melhor organização do processo de compilação (pipeline em fases).**

Etapas da implementação

A solução foi desenvolvida utilizando as ferramentas Flex e Bison, seguindo uma arquitetura de compilador em fases:

1. Análise Léxica (lexer.l)

Responsável por transformar o código fonte ObsAct em tokens.

- **Reconhecimento de palavras-chave:**
 - dispositivo, set, se, entao, senao, ligar, desligar, verificar, enviar alerta
- **Reconhecimento de identificadores (ID)**
- **Reconhecimento de números e strings**
- **Reconhecimento de operadores relacionais e lógicos**

2. Análise Sintática (parser.y)

Responsável por validar a estrutura do programa e construir a AST.

- **Implementação da gramática da linguagem ObsAct.**
- **Construção de nós da AST para:**
 - comandos
 - expressões
 - condições

- estruturas de controle
- Tratamento de:
 - if/else
 - atribuições
 - ações
 - broadcast

3. Árvore Sintática Abstrata (AST)

A AST foi implementada como uma estrutura em C contendo:

- tipo do nó (set, ação, if, lista, etc.)
- valor textual do comando
- ponteiros para filhos (estrutura hierárquica)

Essa estrutura permite representar o programa de forma independente da sintaxe original.

4. Geração de Código (codegen)

A fase final percorre a AST recursivamente e gera código Python.

- Uso de função recursiva gerar()
- Controle de indentação baseado em nível de profundidade da árvore
- Tradução direta dos nós da AST para comandos Python

Gramática Final

A gramática foi adaptada para suportar construção de AST:

PROGRAM → **DEVICES COMMANDS**

DEVICES → **DEVICE DEVICES | DEVICE**

DEVICE → dispositivo : { ID | ID , ID }

COMMANDS → **COMMAND COMMANDS | COMMAND**

COMMAND → **ATTRIB . | ACTION . | ALERTA . | IF**

ATTRIB → set ID = VALUE
| set ID = verificar(ID)

ACTION → ligar ID
| desligar ID
| verificar(ID)

ALERTA → enviar alerta (STRING) ID

IF → se COND entao COMMANDS
| se COND entao COMMANDS senao COMMANDS

COND → ID OP VALUE | COND && COND

VALUE → NUM | BOOL | ID

Acertos e Limitações

O que funciona:

- Análise léxica completa da linguagem ObsAct
- Construção da AST corretamente
- Geração de código Python funcional
- Operadores lógicos e relacionais

Limitações:

- Não há tratamento avançado de erros sintáticos (recuperação limitada)
- Não há verificação de tipos (tudo tratado como string/integer simples)

Casos de teste utilizados

Caso 1 – Inicialização e atribuição

dispositivo : { Termometro , temperatura }
set temperatura = 40.

Resultado: variável inicializada corretamente

Caso 2 – Condicional simples

se temperatura > 30 entao
ligar ventilador.

Resultado: geração de if em Python

Caso 3 – Condicional com else

se movimento == True entao
ligar lampada.
senao
desligar lampada.

Resultado: estrutura if/else correta

Caso 4 – Broadcast

enviar alerta ("Mensagem") para todos : celular, monitor.

Resultado: alertas gerados para múltiplos dispositivos

Caso 5 – Atribuição com verificação

set estado = verificar(ventilador).

Resultado: chamada de função corretamente convertida

Conclusão

O projeto implementa com sucesso um compilador/transpilador da linguagem ObsAct para Python, utilizando técnicas clássicas de compilação baseadas em:

- **Análise léxica**
- **Análise sintática**
- **AST (Árvore Sintática Abstrata)**
- **Geração de código**

A abordagem adotada permite expansão futura da linguagem, incluindo novos comandos e estruturas de controle, mantendo uma arquitetura modular e organizada.